

01 — Numeric Data Types in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Overview of Numeric Types](#)
 3. [Integer Types](#)
 4. [Arbitrary-Precision Types](#)
 5. [Floating-Point Types](#)
 6. [Serial & Identity Types](#)
 7. [Type Hierarchy Diagram](#)
 8. [Storage & Range Reference Table](#)
 9. [SQL Examples](#)
 10. [Overflow Behavior](#)
 11. [Type Casting & Coercion](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Select the correct numeric type for every use-case (counters, money, scientific data, identifiers)
 - Explain storage sizes, value ranges, and precision semantics for every PostgreSQL numeric type
 - Predict and handle overflow, underflow, and rounding behavior
 - Understand the performance trade-offs between integer, decimal, and floating-point arithmetic
 - Use `SERIAL`, `BIGSERIAL`, and `GENERATED AS IDENTITY` correctly
 - Avoid the classic mistakes: using `FLOAT` for money, `SERIAL` for distributed systems, `NUMERIC` where `INTEGER` suffices
-

Overview of Numeric Types

PostgreSQL ships with a rich set of numeric types organized into three families:

```
Numeric Types
├─ Exact types
│  ├── smallint    (2 bytes, integer)
│  ├── integer     (4 bytes, integer)
│  ├── bigint      (8 bytes, integer)
│  ├── numeric(p,s) (variable, arbitrary precision)
│  └── decimal(p,s) (alias for numeric)
├─ Approximate types
│  ├── real        (4 bytes, IEEE 754 single)
│  └── double precision (8 bytes, IEEE 754 double)
└─ Auto-increment helpers
   ├── smallserial (2 bytes)
```

```
|— serial      (4 bytes)
|— bigserial   (8 bytes)
```

The key distinction is **exact vs. approximate**:

- **Exact types** store values with complete precision — no rounding ever occurs.
- **Approximate types** use binary floating-point — rounding is inherent.

Integer Types

smallint (int2)

- **Storage:** 2 bytes
- **Range:** -32,768 to +32,767
- **Use cases:** Status codes, small enumeration columns, age (if you are sure it never exceeds 32 767), flag bitmasks

integer (int4) — the default `int`

- **Storage:** 4 bytes
- **Range:** -2,147,483,648 to +2,147,483,647 (about ± 2.1 billion)
- **Use cases:** Primary keys for most tables, quantities, counters

bigint (int8)

- **Storage:** 8 bytes
- **Range:** -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 (about $\pm 9.2 \times 10^{18}$)
- **Use cases:** High-volume counters, snowflake IDs, Unix timestamps in microseconds, financial transaction IDs

Memory layout (integer – 4 bytes):

sign	31 value bits
------	---------------

 = 2^{31} values on each side of zero

Memory layout (bigint – 8 bytes):

sign	63 value bits
------	---------------

Arbitrary-Precision Types

numeric(precision, scale) / decimal(precision, scale)

`numeric` and `decimal` are **identical** in PostgreSQL — `decimal` is an SQL standard alias.

- **precision (p):** total number of significant digits (both sides of the decimal point), maximum 1000
- **scale (s):** digits to the right of the decimal point
- **Storage:** variable — 2 bytes overhead + ~2 decimal digits per 4-byte group
- **Range:** up to 131,072 digits before decimal point; up to 16,383 digits after

```
numeric(10, 4) – 10 total digits, 4 after decimal
Stores: 123456.7890 ✓
```

```
1234567.890 X (7 digits before decimal + 4 scale = 11 digits > precision
10)
```

Unconstrained numeric : Declaring a column as just `numeric` (no precision) accepts any value without rounding — useful for calculations where you want maximum fidelity.

Performance note: `numeric` arithmetic is done in software, not hardware — it is ~10-100× slower than integer arithmetic for CPU-intensive workloads.

Money type (special case)

PostgreSQL has a `money` type but it is **not recommended** — it is locale-dependent and causes portability problems. Use `numeric(19,4)` instead.

Floating-Point Types

real (float4)

- **Storage:** 4 bytes (IEEE 754 single-precision)
- **Precision:** ~6 significant decimal digits
- **Range:** $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ plus special values: `Infinity` , `-Infinity` , `NaN`

double precision (float8)

- **Storage:** 8 bytes (IEEE 754 double-precision)
- **Precision:** ~15 significant decimal digits
- **Range:** $\pm 2.23 \times 10^{-308}$ to $\pm 1.8 \times 10^{308}$
- **Aliases:** `float` , `float8` , `double precision`

Why floating-point is inexact:

```
SELECT 0.1::float + 0.2::float = 0.3::float;
-- Result: FALSE ← classic IEEE 754 gotcha

SELECT 0.1::numeric + 0.2::numeric = 0.3::numeric;
-- Result: TRUE ← numeric is exact
```

Use cases for floating-point: Scientific measurements, statistical calculations, machine-learning features, geographic coordinates, any domain where a tiny rounding error is acceptable and speed matters.

Serial & Identity Types

SERIAL family (legacy — pre-PostgreSQL 10)

`SERIAL` , `SMALLSERIAL` , and `BIGSERIAL` are **not real types** — they are shorthand macros that:

1. Create a sequence object
2. Set the column default to `nextval('sequence_name')`
3. Add a `NOT NULL` constraint

```
-- This:
CREATE TABLE t (id SERIAL PRIMARY KEY);
```

```
-- Expands to:
CREATE SEQUENCE t_id_seq;
CREATE TABLE t (
    id INTEGER NOT NULL DEFAULT nextval('t_id_seq')
);
ALTER SEQUENCE t_id_seq OWNED BY t.id;
```

Problems with SERIAL:

- The sequence is a separate object — `DROP TABLE` does not drop it unless owned
- Permissions on the sequence must be managed separately
- Does not comply with SQL standard

GENERATED AS IDENTITY (recommended — PostgreSQL 10+)

```
CREATE TABLE orders (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    amount NUMERIC(12,2) NOT NULL
);

-- ALWAYS: rejects manual inserts/updates to id
-- BY DEFAULT: allows manual override (useful for migrations)
CREATE TABLE legacy_import (
    id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
);
```

Custom sequence options:

```
CREATE TABLE events (
    id BIGINT GENERATED ALWAYS AS IDENTITY
    (START WITH 1000 INCREMENT BY 1 CACHE 50)
    PRIMARY KEY
);
```

Type Hierarchy Diagram

NUMERIC TYPE HIERARCHY

Integer Family	Arbitrary Precision	Floating Point Family
smallint 2 bytes	numeric(p,s) variable bytes	real 4 bytes
integer 4 bytes	decimal(p,s) (alias)	double precision 8 bytes

bigint
8 bytes

EXACT ← → APPROXIMATE
(no rounding) (IEEE 754)

Storage & Range Reference Table

Type	Bytes	Min Value	Max Value	Precision
smallint	2	-32,768	+32,767	Exact
integer	4	-2,147,483,648	+2,147,483,647	Exact
bigint	8	-9.2×10^{18}	$+9.2 \times 10^{18}$	Exact
decimal/numeric	var	-10^{131072}	$+10^{131072}$	Exact (any p,s)
real	4	-3.4×10^{38}	$+3.4 \times 10^{38}$	~6 digits
double precision	8	-1.8×10^{308}	$+1.8 \times 10^{308}$	~15 digits
smallserial	2	1	32,767	Exact
serial	4	1	2,147,483,647	Exact
bigserial	8	1	9.2×10^{18}	Exact

SQL Examples

Creating tables with numeric types

```
-- Product catalog: typical mix of numeric types
CREATE TABLE products (
  product_id    INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku           VARCHAR(50)  NOT NULL UNIQUE,
  name          VARCHAR(255) NOT NULL,
  quantity_on_hand SMALLINT  NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
  reorder_level SMALLINT     NOT NULL DEFAULT 10,
  unit_price    NUMERIC(12, 4) NOT NULL CHECK (unit_price > 0),
  discount_pct  NUMERIC(5, 2)  CHECK (discount_pct BETWEEN 0 AND 100),
  weight_kg     REAL,
  rating        REAL           CHECK (rating BETWEEN 0.0 AND 5.0),
  total_sold    BIGINT         NOT NULL DEFAULT 0,
  created_at    TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- Scientific measurements: use double precision
CREATE TABLE sensor_readings (
  reading_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sensor_id     INTEGER NOT NULL,
  temperature   DOUBLE PRECISION NOT NULL, -- scientific precision needed
  pressure      DOUBLE PRECISION NOT NULL,
  humidity      REAL,                      -- single precision is fine here
```

```

    recorded_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Financial table: always use numeric for money
CREATE TABLE ledger_entries (
    entry_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_id        INTEGER NOT NULL,
    debit_amount       NUMERIC(19, 4) NOT NULL DEFAULT 0,
    credit_amount      NUMERIC(19, 4) NOT NULL DEFAULT 0,
    running_balance    NUMERIC(19, 4) NOT NULL,
    entry_date         DATE NOT NULL
);

```

Numeric operations and functions

```

-- Rounding functions
SELECT
    round(3.14159, 2) AS round_2dp,      -- 3.14
    ceil(3.1)         AS ceiling,        -- 4
    floor(3.9)        AS floor_val,      -- 3
    trunc(3.9)        AS truncated,      -- 3
    abs(-42)          AS absolute,       -- 42
    mod(17, 5)        AS modulo,         -- 2
    power(2, 10)       AS two_to_ten,    -- 1024
    sqrt(144)         AS square_root,    -- 12
    factorial(10)      AS ten_factorial,  -- 3628800
    gcd(48, 18)       AS greatest_common_divisor; -- 6 (PostgreSQL 13+)

-- Type casting
SELECT
    42::BIGINT,          -- integer cast to bigint
    3.14::INTEGER,       -- truncates to 3 (NOT rounded)
    '123.45'::NUMERIC(10,2), -- string to numeric
    123::NUMERIC / 7::NUMERIC; -- exact division: 17.571428571428571429

-- Aggregate numeric functions
SELECT
    sum(unit_price) AS total,
    avg(unit_price) AS average,
    min(unit_price) AS cheapest,
    max(unit_price) AS most_expensive,
    stddev(unit_price) AS std_deviation,
    variance(unit_price) AS variance_val
FROM products;

```

Overflow Behavior

Integer overflow in PostgreSQL raises an **error** — it does not wrap silently.

```
-- Integer overflow: raises ERROR
SELECT 2147483647::INTEGER + 1;
-- ERROR: integer out of range

-- Bigint is safe for very large values
SELECT 2147483647::BIGINT + 1;
-- 2147483648

-- numeric never overflows within its declared precision
SELECT 9999999999::NUMERIC + 1;
-- 10000000000

-- Serial exhaustion: inserting beyond the max silently fails
-- with "nextval of sequence ... exceeded" error
-- Solution: use BIGSERIAL or BIGINT GENERATED AS IDENTITY
```

Division behavior:

```
SELECT 7 / 2;           -- returns 3 (integer division, truncates)
SELECT 7.0 / 2;         -- returns 3.5 (at least one operand is numeric)
SELECT 7::FLOAT / 2;    -- returns 3.5 (float division)
SELECT 7 % 2;           -- returns 1 (modulo)
SELECT -7 / 2;          -- returns -3 (truncates toward zero in PostgreSQL)
```

Type Casting & Coercion

```
-- Implicit promotion rules (narrower → wider):
-- smallint → integer → bigint → numeric → double precision

-- Explicit casting
SELECT CAST(3.7 AS INTEGER); -- 3 (truncates, not rounds!)
SELECT 3.7::INTEGER;         -- 3
SELECT INTEGER '42';         -- 42 (type constructor syntax)

-- Safe division with nullif to prevent divide-by-zero
SELECT revenue / NULLIF(units_sold, 0) AS avg_revenue_per_unit
FROM sales;

-- Converting currency strings
SELECT REPLACE('$1,234.56', '$', '')::NUMERIC; -- 1234.56 after removing comma too
```

Common Mistakes

1. Using `FLOAT` or `REAL` for monetary amounts

```
-- BAD: floating-point rounding causes penny errors
total_price REAL
```

```
-- GOOD: exact decimal arithmetic
total_price NUMERIC(12, 4)
```

2. Using `INTEGER` for high-volume identity columns

```
-- BAD: will exhaust ~2.1 billion values in busy systems
id SERIAL PRIMARY KEY

-- GOOD:  $9.2 \times 10^{18}$  values — effectively inexhaustible
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

3. Confusing `round()` and cast to integer

```
SELECT round(3.5)::INTEGER; -- 4 (rounds then casts)
SELECT 3.5::INTEGER;        -- 3 (truncates, no rounding!)
```

4. Not specifying precision for `NUMERIC` in critical columns

```
-- Risky: unconstrained numeric accepts any precision
price NUMERIC

-- Better: enforce domain constraints at the type level
price NUMERIC(12, 4) CHECK (price > 0)
```

5. Using `SMALLINT` for codes that may grow — Status codes, HTTP codes, product categories often outgrow 32 767.

6. Forgetting that `serial` gaps are normal — Rolled-back transactions consume sequence values. Never treat `SERIAL` gaps as errors.

Best Practices

1. Default to `INTEGER` for PKs in small/medium tables; use `BIGINT` for any table expected to grow beyond 1 billion rows or in distributed systems.
 2. Always use `NUMERIC(p,s)` for money — never `FLOAT`, never `REAL`, never the `money` type.
 3. Prefer `GENERATED AS IDENTITY` over `SERIAL` for new code.
 4. Use `SMALLINT` only when you are certain of the range — saves 2 bytes per row vs. `INTEGER` but adds risk.
 5. Use `DOUBLE PRECISION` for scientific/statistical data; use `REAL` only when storage is critically constrained.
 6. Add `CHECK` constraints to numeric columns to enforce domain rules (`CHECK (price > 0)` , `CHECK (pct BETWEEN 0 AND 100)`).
 7. Document units in column comments: `COMMENT ON COLUMN sensor_readings.temperature IS 'Temperature in Celsius' .`
-

Performance Considerations

Operation speed (approximate, single-row):

```
INTEGER arithmetic   : ~1 ns   (CPU instruction)
BIGINT arithmetic    : ~1-2 ns (CPU instruction on 64-bit)
FLOAT/DOUBLE arith.  : ~2-4 ns (FPU instruction)
NUMERIC arithmetic   : ~50-500 ns (software emulation, depends on precision)
```

- **Storage size impacts index size and cache utilization.** Prefer the smallest correct type.
- `numeric` columns with high precision cause wider index entries → fewer index entries per page → more I/O.
- Sorting `INTEGER` columns is faster than sorting `NUMERIC` columns.
- For OLAP aggregations on large tables, `BIGINT SUM` is significantly faster than `NUMERIC SUM`.
- Consider `REAL` for machine-learning feature columns where minor precision loss is acceptable and you need compact storage.

Index size comparison (1 million rows):

```
SMALLINT index : ~27 MB
INTEGER index  : ~35 MB
BIGINT index   : ~45 MB
NUMERIC(10,4)  : ~60 MB (variable encoding overhead)
```

Interview Questions & Answers

Q1: What is the difference between `numeric` and `decimal` in PostgreSQL?

A: They are identical — `decimal` is an SQL standard alias for `numeric`. Both store arbitrary-precision exact decimal numbers. PostgreSQL implements them as the same type internally.

Q2: Why should you never use `FLOAT` or `REAL` to store monetary values?

A: IEEE 754 floating-point arithmetic cannot exactly represent many decimal fractions (e.g., 0.1 has no exact binary representation). This causes rounding errors that accumulate over calculations. For example, $0.1 + 0.2 \neq 0.3$ in floating-point. Financial calculations require exact arithmetic — use `NUMERIC(p,s)` instead.

Q3: What happens when a `SERIAL` column runs out of values?

A: PostgreSQL raises an error: `nextval of sequence exceeded maximum value`. The table itself is unaffected. You must `ALTER SEQUENCE ... MAXVALUE` or migrate the column to `BIGSERIAL` / `BIGINT IDENTITY`. For new systems, always use `BIGINT GENERATED AS IDENTITY` to avoid this.

Q4: What is the difference between `SERIAL` and `GENERATED AS IDENTITY`?

A: `SERIAL` is a PostgreSQL-specific macro that creates a sequence and sets a default. `GENERATED AS IDENTITY` is SQL standard (SQL:2003), gives finer control over sequence parameters, and supports `ALWAYS` (rejects manual inserts) vs `BY DEFAULT` modes. `GENERATED ALWAYS` prevents accidental overwrites of identity values.

Q5: What does `integer / integer` return in PostgreSQL?

A: Integer (truncated toward zero). $7 / 2 = 3$, not 3.5. To get decimal division, cast at least one operand: `7.0 / 2` or `7::NUMERIC / 2`.

Q6: What is the maximum value of a `BIGINT`?

A: 9,223,372,036,854,775,807 ($2^{63} - 1$, approximately 9.2×10^{18}).

Q7: When would you choose `SMALLINT` over `INTEGER` ?

A: When the column is guaranteed to stay within $-32,768$ to $+32,767$ AND the table has very high row counts where the 2-byte savings per row matters (e.g., a trillion-row table saves ~ 2 TB). In practice, the risk of outgrowing `SMALLINT` usually outweighs the storage saving.

Q8: What are the special values for floating-point types?

A: `'Infinity'`, `'-Infinity'`, and `'NaN'` (Not a Number). NaN comparisons are special: `NaN = NaN` returns `TRUE` in PostgreSQL (unlike IEEE 754 standard where `NaN  $\neq$  NaN`).

Q9: How does PostgreSQL handle numeric overflow differently from many other languages?

A: PostgreSQL raises an ERROR rather than wrapping around. This is safer for database integrity — you get a clear failure rather than silent data corruption.

Q10: What is the storage cost of `NUMERIC(p,s)` vs `INTEGER` ?

A: `INTEGER` is always exactly 4 bytes. `NUMERIC` has 2 bytes of overhead plus approximately 4 bytes per 4-digit group. A `NUMERIC(10,2)` column might use 8–12 bytes per value depending on the actual stored digits.

Exercises with Solutions

Exercise 1

Create a `bank_accounts` table where the balance must be `NUMERIC` with 2 decimal places, must be ≥ 0 , and the account ID should use the modern identity syntax.

Solution:

```
CREATE TABLE bank_accounts (  
    account_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    account_number  VARCHAR(20) NOT NULL UNIQUE,  
    holder_name     VARCHAR(255) NOT NULL,  
    balance         NUMERIC(15, 2) NOT NULL DEFAULT 0.00 CHECK (balance >= 0),  
    account_type    SMALLINT    NOT NULL DEFAULT 1, -- 1=checking, 2=savings  
    opened_date     DATE        NOT NULL DEFAULT CURRENT_DATE,  
    is_active       BOOLEAN     NOT NULL DEFAULT TRUE  
);
```

Exercise 2

Write a query that demonstrates the floating-point precision problem with a concrete example, then fix it using `NUMERIC`.

Solution:

```
-- Demonstrate the problem  
SELECT  
    0.1::FLOAT8 + 0.2::FLOAT8 AS float_result, --  
    0.30000000000000004
```

```
(0.1::FLOAT8 + 0.2::FLOAT8) = 0.3::FLOAT8 AS float_equal,      -- FALSE
0.1::NUMERIC + 0.2::NUMERIC AS numeric_result,                  -- 0.3
(0.1::NUMERIC + 0.2::NUMERIC) = 0.3::NUMERIC AS numeric_equal;  -- TRUE
```

Exercise 3

What query would you use to find the safe integer range for a column and determine if it should be upgraded from `INTEGER` to `BIGINT` ?

Solution:

```
SELECT
  min(order_id)           AS min_id,
  max(order_id)           AS max_id,
  max(order_id)::FLOAT / 2147483647 * 100 AS pct_of_int_max,
  CASE
    WHEN max(order_id) > 1500000000 THEN 'URGENT: Migrate to BIGINT'
    WHEN max(order_id) > 1000000000 THEN 'WARNING: Consider BIGINT soon'
    ELSE 'OK for now'
  END AS recommendation
FROM orders;
```

Exercise 4

Compute the compound interest on a principal of \$10,000 at 3.5% annual rate for 5 years using exact arithmetic.

Solution:

```
SELECT
  10000::NUMERIC(12,4) *
  POWER(1 + 0.035::NUMERIC(8,6), 5)::NUMERIC(12,8)
  AS compound_interest_future_value;
-- Returns: 11876.8635 (exact to 4 decimal places)
```

Cross-References

- `02_character_types.md` — analogous discussion for text storage
- `05_json_jsonb.md` — numeric values inside JSON documents
- `08_constraints.md` — CHECK constraints on numeric columns
- `09_sequences_identity.md` — deep dive into sequences and identity columns
- `../07_Indexes/` — indexing strategies for numeric columns
- `../08_Query_Optimization/` — query plans for numeric comparisons